



Kubernetes (K8s)



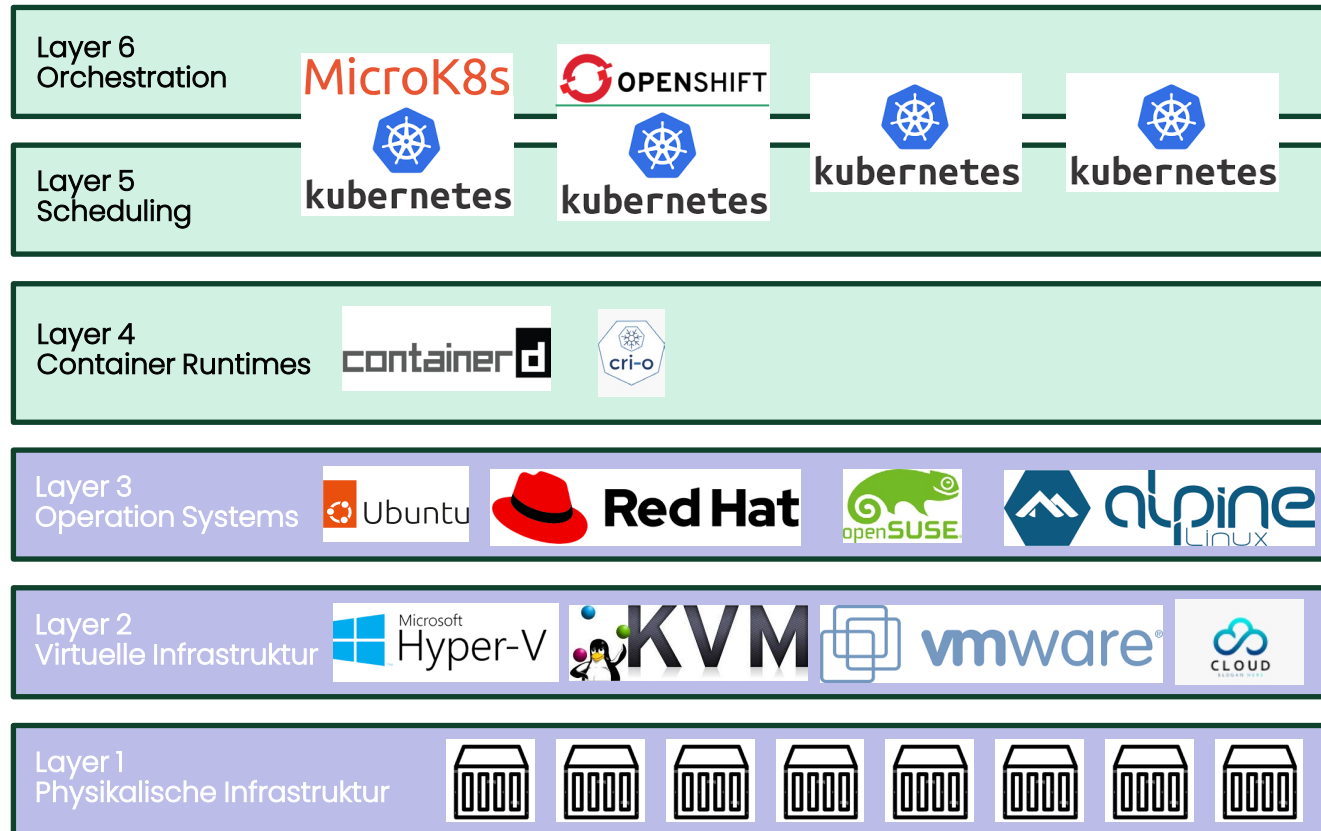
Lernziele

- Sie haben einen ersten Einblick in Kubernetes.
- Sie haben einen ersten Überblick über das Container Ökosystem.

Zeitlicher Ablauf

- Die (vereinfachten) »Layer« der Container-Welt, Layer 5 und 6
- Kubernetes (K8s) im Überblick
- Wer steht hinter Kubernetes?
- Was heisst Kubernetes Zertifiziert?
- Kubernetes (K8s) Merkmale
- K8s-Komponenten
- K8s Control Plane- (alt Master) und Worker-Setup
- K8s-Dienste auf der Control Plane und Worker Nodes
- Networking in Kubernetes
- Reflexion
- Lernzielkontrolle

Was macht Kubernetes?



- **Orchestrierung:** Ausführung eines definierten Workflows: zuerst A, dann B, dann C.
- **Scheduling:** Zuweisung / Verteilung der Pods/Container auf die Nodes (VMs) innerhalb des Kubernetes Cluster.

Die (vereinfachten) »Layer« der Container-Welt

Was macht Kubernetes?

	Canonical	RedHat	SuSE	Dist. X
Layer 6 Orchestration	MicroK8s	OPENSIFT	RANCHER	
Layer 5 Scheduling	kubernetes	kubernetes	kubernetes	kubernetes
Layer 4 Container Runtimes	containerd	cri-o	containerd	containerd
Layer 3 Operation Systems	Ubuntu	Red Hat	openSUSE	XXX
Layer 2 Virtuelle Infrastruktur	Microsoft Hyper-V	KVM	vmware	CLOUD
Layer 1 Physikalische Infrastruktur				

Die (vereinfachten) »Layer« der Container-Welt

<https://www.cncf.io/certification/software-conformance/>

- Kubernetes ist eine Reihe von **unabhängigen, komponierbaren Steuerungsprozessen** die zu unterschiedlichen **Distributionen** zusammengefasst sind.



Wie wird die Kompatibilität/Austauschbarkeit geprüft?

Zertifizierte Kubernetes Softwarekonformität

- Die Softwarekonformität stellt sicher, dass die Kubernetes-Version jedes Anbieters die erforderlichen APIs unterstützt, ebenso wie Open-Source-Community-Versionen.

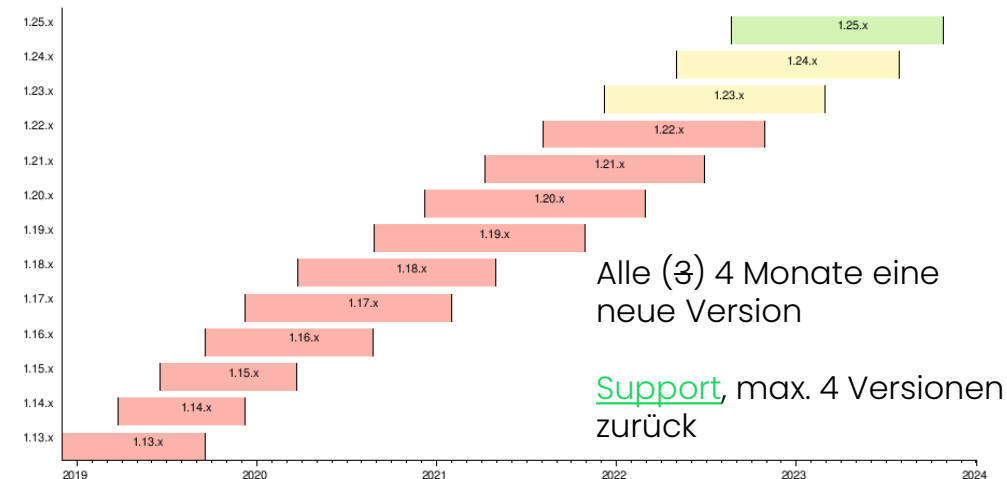
Kubernetes Conformance Program

- **Konsistenz** : Benutzer möchten Konsistenz bei der Interaktion mit einer Installation von Kubernetes.
- **Rechtzeitige Updates** : Um zertifiziert zu bleiben, müssen Anbieter die neueste Version von Kubernetes jährlich oder häufiger bereitstellen, sodass Sie sicher sein können, dass Sie immer Zugriff auf die neuesten Funktionen haben, an deren Bereitstellung die Community hart gearbeitet hat.
- **Bestätigbarkeit** : Jeder Endbenutzer kann bestätigen, dass seine Distribution oder Plattform konform bleibt, indem er die identische Open-Source-Konformitätsanwendung ([Sonobuoy](#)) ausführt, die zur Zertifizierung verwendet wurde.

Kubernetes (K8s) im Überblick



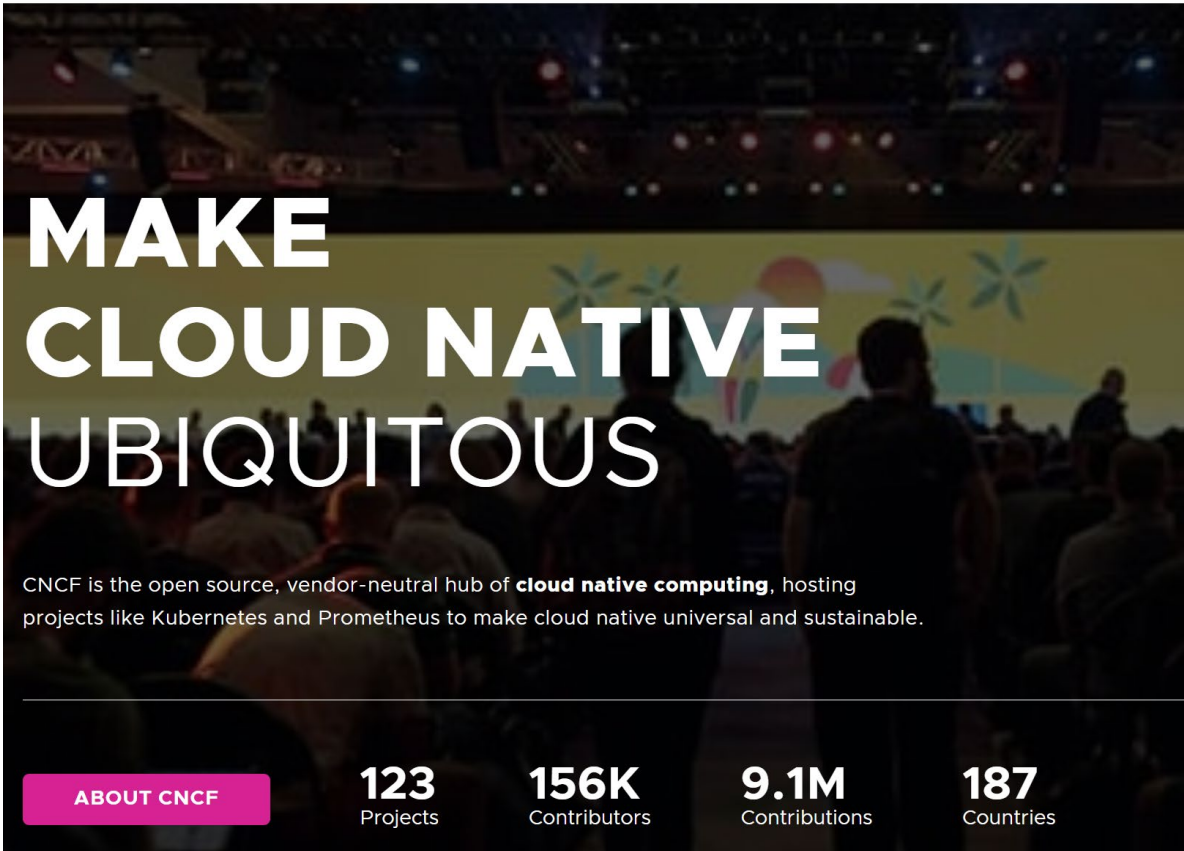
- Das im Juli 2014 gestartete **Kubernetes** (griechisch: Steuermann) stellt die derzeit **populärste** Container-Cluster-/Orchestrierungs-Lösung dar.
- Kubernetes wurde von **Google** ursprünglich als Orchestrierungs-Framework unter der Code-Bezeichnung [Borg](#) initiiert. **Ziel** war es, Docker- oder Rocket-(CoreOS)-**Container** im grossen Umfang zu **deployen**, zu **administrieren** und transparent zu **orchestrieren**.
- Kubernetes' Hauptaufgabe ist die **Verwaltung und Orchestrierung der Container** innerhalb eines **Clusters**, der üblicherweise aus mindestens einem Kubernetes **Control Plane** und **multiplen Worker Nodes** besteht.
- K8s **arbeitete** primär mit Docker-Containern und APIs (Kubernetes ist [deprecating Docker](#), per 1.1.21, EOL 1.5.22)
- Kubernetes ist mittlerweile bei der **Cloud Native Computing Foundation (CNCF)** gehostet.



Wer steht hinter Kubernetes?


CLOUD NATIVE
COMPUTING FOUNDATION

[About](#)
[Projects](#)
[Training](#)
[Community](#)
[Blog & News](#)



MAKE CLOUD NATIVE UBIQUITOUS

CNCF is the open source, vendor-neutral hub of **cloud native computing**, hosting projects like Kubernetes and Prometheus to make cloud native universal and sustainable.

[ABOUT CNCF](#)

123
Projects

156K
Contributors

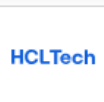
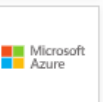
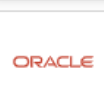
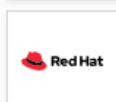
9.1M
Contributions

187
Countries

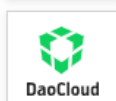
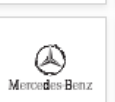
<https://www.cncf.io/about/members/>

CNCF Members

Platinum (18)

Gold (27)

CNCF-Philosophie – Projekte

Our graduated projects

[VIEW ON CNCF LANDSCAP](#)

The grid displays 12 CNCF graduated projects with their logos and names:

- Container Runtime**: Logo with a square and 'd'.
- CoreDNS**: Logo with a purple cube.
- envoy**: Logo with three pink cubes.
- etcd**: Logo with a blue gear and 'etcd' text.
- fluentd**: Logo with a blue bird.
- HARBOR**: Logo with a green lighthouse.
- HELM**: Logo with a blue ship's wheel.
- JAEGER**: Logo with a blue cartoon character.
- kubernetes**: Logo with a blue ship's wheel.
- LINKERD**: Logo with a green cube.
- Open Policy Agent**: Logo with a blue shield.
- Viteess**: Logo with an orange 'V'.

Stufe	Stabilität	Einsatz in der Praxis	Community-Beteiligung	Sicherheitsbewusstsein	Beispielprojekte
Sandbox	Experimentell	Sehr begrenzt oder noch keine	Gering, wenige Mitwirkende	Keine oder kaum vorhanden	Neuere Projekte
Incubating	Reif, aber nicht 100% stabil	Erste Produktionseinsätze	Wachsende Community	Erste Sicherheitsprüfungen	Argo, OpenTelemetry
Graduated	Hoch stabil	Breite Produktionseinsätze	Große, diversifizierte Community	Vollständig ausgereiftes Sicherheitskonzept	Kubernetes, Prometheus

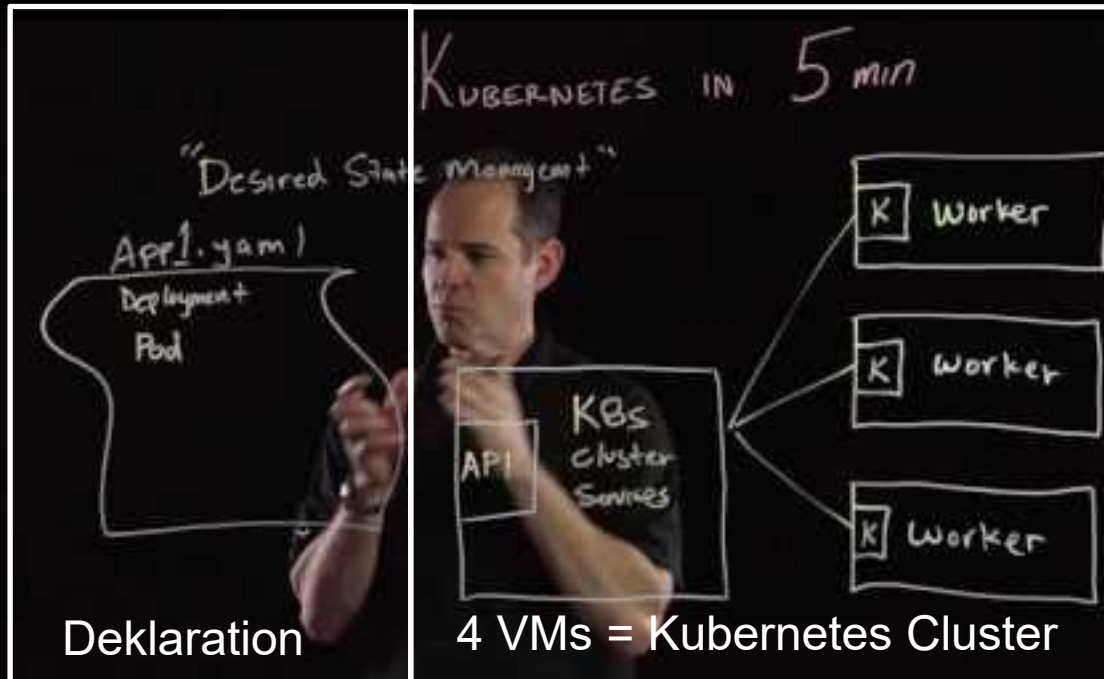
- Ein Teil der CNCF-Philosophie besteht darin, eine Reihe sich ergänzender Projekte und Tools bereitzustellen, die für die Arbeit mit Cloud Computing entwickelt wurden.
- Cloud-native Projekte funktionieren gut zusammen, wenn sie darauf ausgelegt sind, ein einzelnes, klar definiertes Problem zu lösen.
- Das bedeutet, dass andere Projekte frühere Innovationen ergänzen und erweitern können, um schnell voranzukommen.
- <https://www.cncf.io/projects/>

Was ist das Ziel der CNCF?

CNCF Cloud Native Definition v1.0

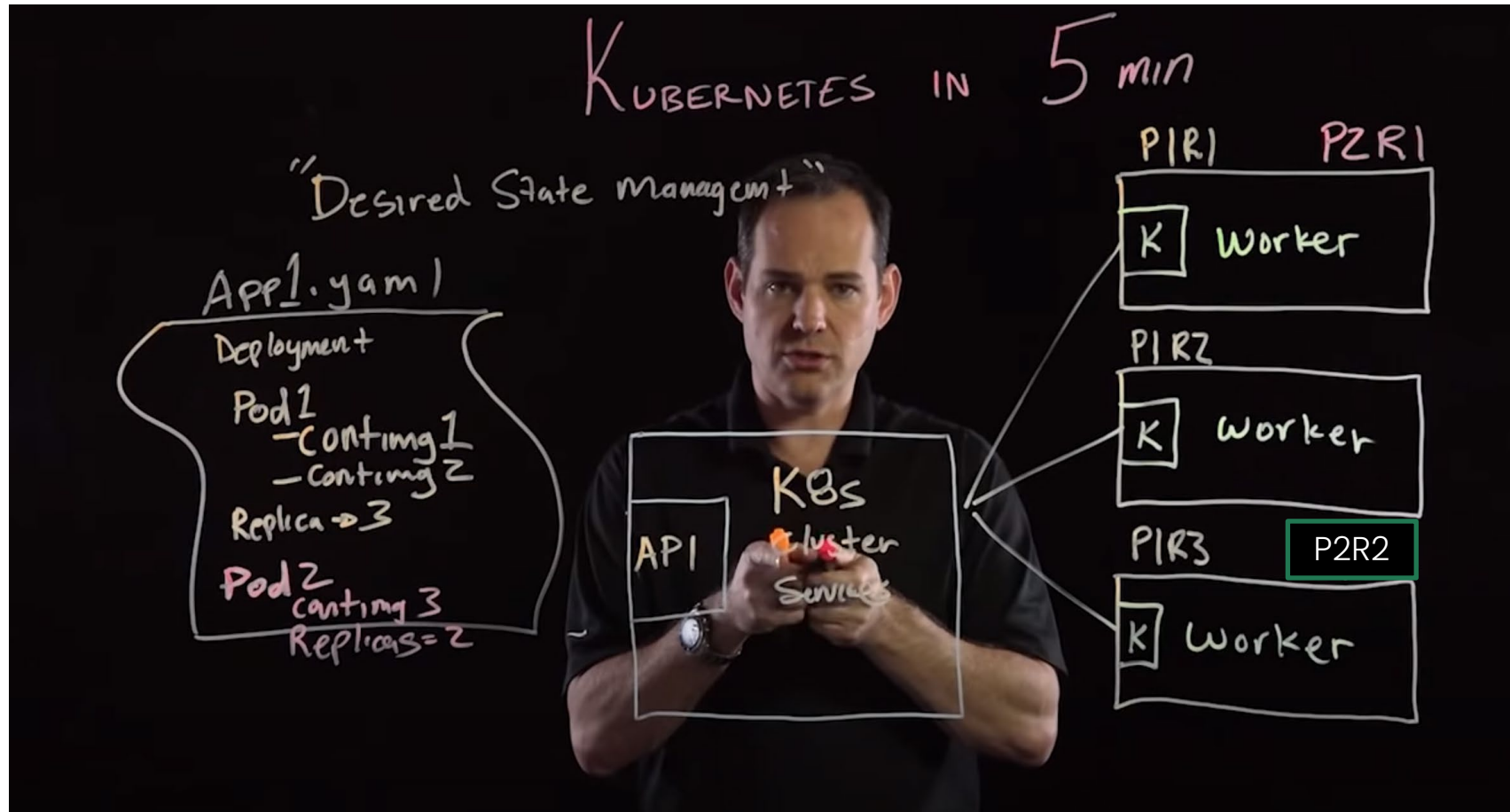
- Cloud native Technologien ermöglichen es Unternehmen, skalierbare Anwendungen in modernen, dynamischen Umgebungen zu implementieren und zu betreiben. Dies können öffentliche, private und Hybrid-Clouds sein. Best-Practises, wie Container, Service-Meshs, Microservices, immutable Infrastruktur und deklarative APIs, unterstützen diesen Ansatz.
- Die zugrundeliegenden Techniken ermöglichen die Umsetzung von entkoppelten Systemen, die belastbar, handhabbar und beobachtbar sind. Kombiniert mit einer robusten Automatisierung können Softwareentwickler mit geringem Aufwand flexibel und schnell auf Änderungen reagieren.
- Die **Cloud Native Computing Foundation** fördert die Akzeptanz dieser Paradigmen durch die Ausgestaltung eines Open Source Ökosystems aus herstellerneutralen Projekten. Wir demokratisieren modernste und innovative Softwareentwicklungs-Patterns, um diese Innovationen für alle zugänglich zu machen.
- Quelle: <https://github.com/cncf/toc/blob/master/DEFINITION.md>
- Willkommen bei Cloud Land! Eine illustrierte Einführung in die [Cloud Native Landscape](#).
- [Cloud-native](#) – was steckt hinter dem Buzzword?
- [Cloud Native Maturity Model](#) (Reifegradmodell)

Kubernetes: in 5 Minuten

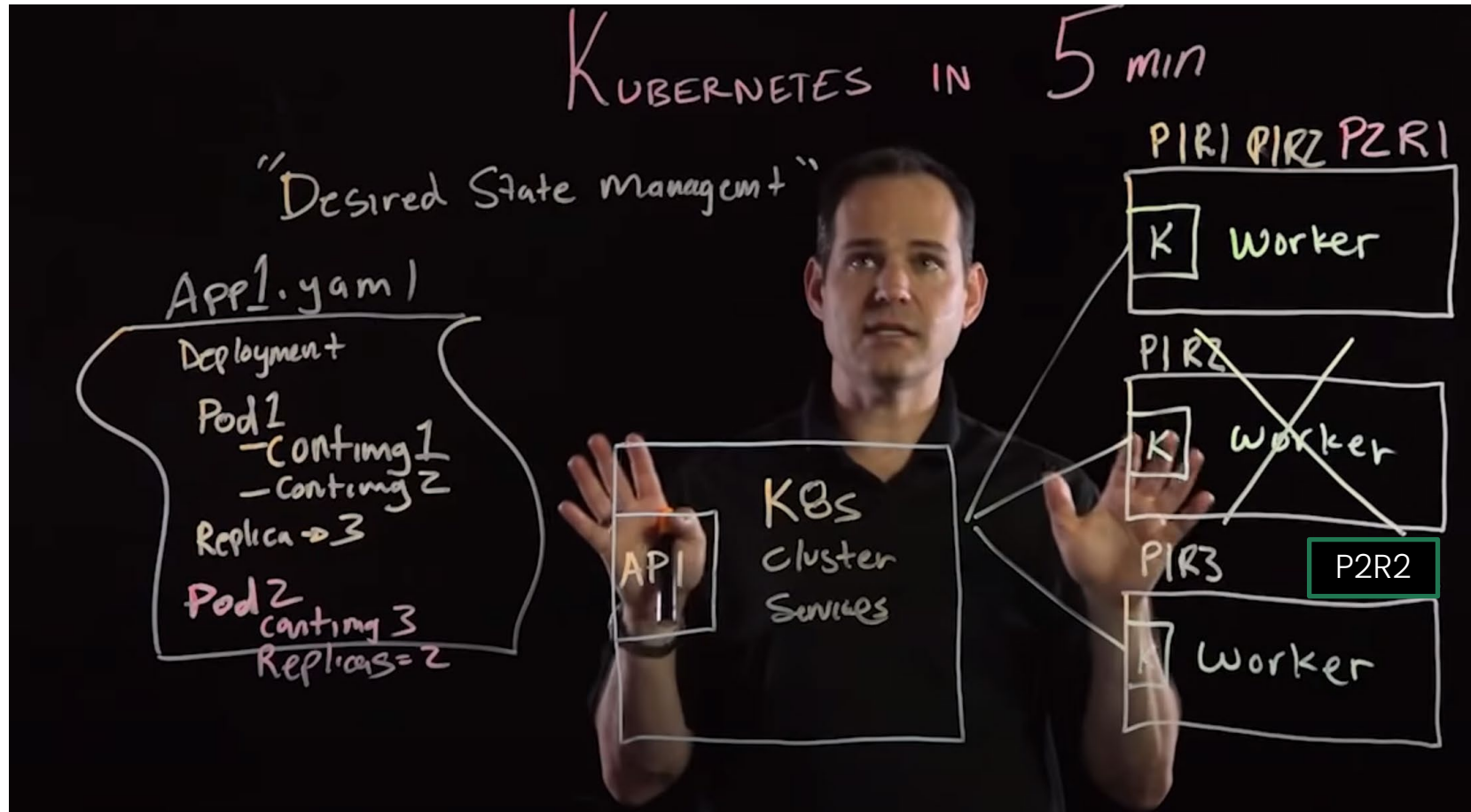


- **Cluster** (vom Englischen für "Rechner-Schwarm", "-Gruppe" oder "-Haufen"), bezeichnet eine Anzahl von vernetzten Computern oder Virtuellen Maschinen.
- Zusammenfassend für zwei unterschiedliche Aufgaben:
 - ◇ die Erhöhung der Rechenkapazität
 - ◇ die Erhöhung der Verfügbarkeit (HA-Cluster, engl. high available - hochverfügbar).
- Die in einem Kubernetes Cluster befindlichen Einheiten werden als (Control Plane oder Worker) Nodes bezeichnet.

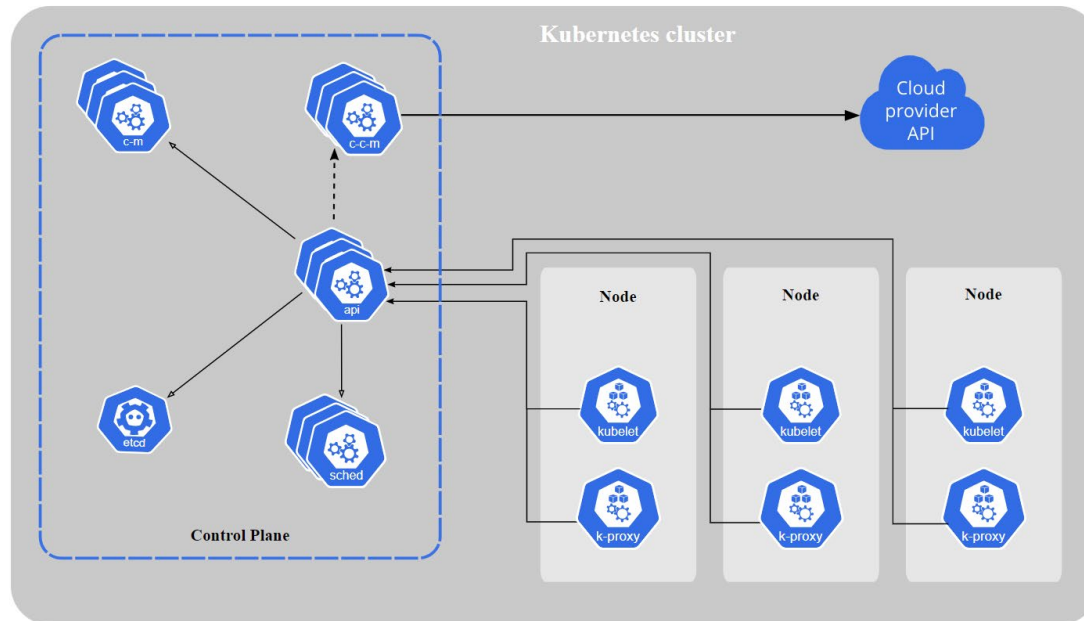
Kubernetes: in 5 Minuten (Deklarativ, API)



Kubernetes: in 5 Minuten (HA – Erhöhung der Verfügbarkeit)

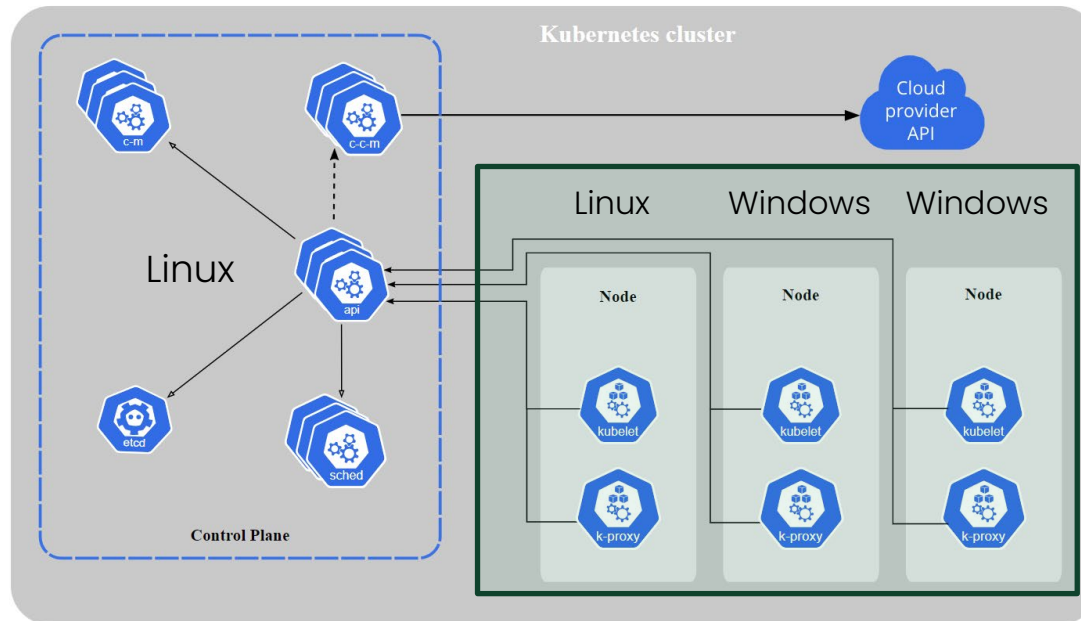


K8s-Komponenten (unterschiedlich pro Distribution)



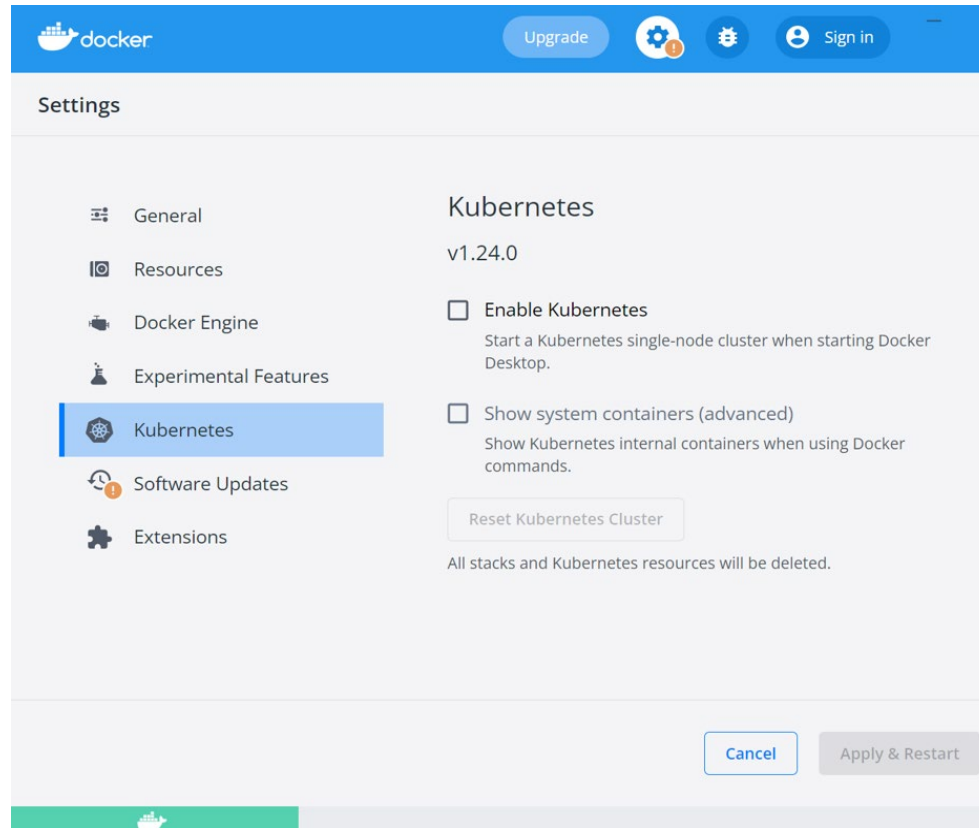
- **Control Plane Nodes (alt Master)**
 - ◇ der zentrale Koordinator in einem K8s Cluster, über den der Cluster gesteuert wird.
- **Worker-Nodes**
 - ◇ Sie sind die Arbeits-Nodes des K8s. Auf ihnen laufen die Container bzw. im K8s-Kontext: Pods
- **Hierarchie:**
 - ◇ Nodes (Control Plane oder Worker) – VMs oder Bare Metal oder Raspberry Pi
 - Node (auch Container Host)
 - ◇ Pod (IP-Adresse)
Containers (Mehrzahl!)
 - Prozess(e)

Windows nur Kubernetes Worker!

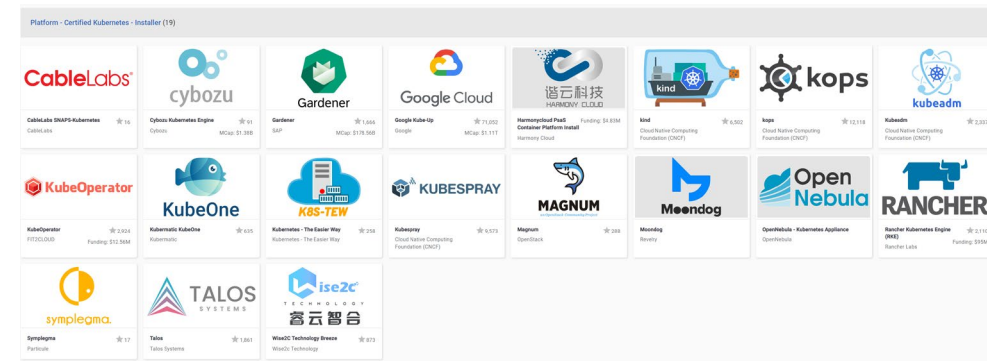


- Um Windows-Container ausführen zu können, muss Ihr Kubernetes-Cluster mehrere Betriebssysteme umfassen, wobei die **Control Planes** (alt Master) **Linux** ausführen und die Worker entweder Windows oder Linux ausführen.
- **Windows Server 2019** war das erste **unterstützte** Windows-Betriebssystem.
- Quelle: [Intro to Windows support in Kubernetes](https://kubernetes.io/docs/concepts/overview/components/)

Welche Installer gibt es für Kubernetes (K8s)?



- [Minikube](#) zum schnell K8s Luft Schnuppern. Nur Single Node, besser [kind](#)
- «**Docker Desktop**»: für Entwickler (siehe links).
- [MicroK8s](#) (Ubuntu) ist das kleinste, schnellste und vollständig konforme Kubernetes System.
- [kubeadm](#): Single Node bis HA Cluster
- Zertifizierte Kubernetes [Installer](#).



<https://www.docker.com/products/docker-desktop/>

Kubernetes Installation mittels kubeadm (vereinfacht)

- Container Runtime

```
$ sudo apt-get update
$ sudo apt-get install docker-ce docker-ce-cli containerd.io
```

```
## Install containerd
```

```
sudo apt-get update && sudo apt-get install -y containerd.io
```

```
sudo dnf module enable cri-o:$VERSION
sudo dnf install cri-o
```

- Kubernetes (Control Plane + Worker)

- ◇ apt-get install -y kubelet kubeadm
- ◇ kubeadm init
- ◇ kubectl apply -f kube-flannel.yaml
- ◇ kubectl apply -f persistentvolumes..
- ◇ kubectl apply -f <weitere>.yaml

- Die einzelnen Schritte für Control Plane und Worker sind wie folgt:

- ◇ Installation Linux z.B. Ubuntu
- ◇ Installation Container Runtime (docker, containerd, cri-o)
- ◇ Installation K8s Komponenten

- Nur auf der Control Plane

- ◇ [kubeadm](#) Initialisieren
- ◇ Overlay (Pods) Netzwerk starten
- ◇ Persistent Volumes (Storage) einrichten
- ◇ Weitere Services wie ingress, Dashboard etc. Starten

Kubernetes Cluster erstellen – Azure Cloud

Create Kubernetes cluster ...

Basics Node pools Access Networking Integrations Advanced Tags Review + create

Azure Kubernetes Service (AKS) manages your hosted Kubernetes environment, making it quick and easy to deploy and manage containerized applications without container orchestration expertise. It also eliminates the burden of ongoing operations and maintenance by provisioning, upgrading, and scaling resources on demand, without taking your applications offline. [Learn more about Azure Kubernetes Service](#)

Project details

Select a subscription to manage deployed resources and costs. Use resource groups like folders to organize and manage all your resources.

Subscription * ⓘ Azure für Bildungseinrichtungen ▼

Resource group * ⓘ (New) Resource group ▼

[Create new](#)

Cluster details

Cluster preset configuration Standard (\$\$) ▼

To quickly customize your Kubernetes cluster, choose one of the preset configurations above. You can modify these configurations at any time. [Learn more and compare presets](#)

Kubernetes cluster name * ⓘ

Region * ⓘ (Europe) West Europe ▼

Availability zones ⓘ Zones 1,2,3 ▼

High availability is recommended for standard configuration.

Kubernetes version * ⓘ 1.23.8 (default) ▼

API server availability ⓘ

☒ 99.95%
Optimize for availability.

☐ 99.5%
Optimize for cost.

99.95% API server availability is recommended for standard configuration.

Primary node pool

The number and size of nodes in the primary node pool in your cluster. For production workloads, at least 3 nodes are recommended for resiliency. For development or test workloads, only one node is required. If you would like to add additional node pools or to see additional configuration options for this node pool, go to the 'Node pools' tab above. You will be able to add additional node pools after creating your cluster. [Learn more about node pools in Azure Kubernetes Service](#)

Node size * ⓘ

Standard DS2 v2

Standard DS2_v2 is recommended for standard configuration. [Change size](#)

Scale method * ⓘ

☐ Manual

☒ Autoscale

Autoscaling is recommended for standard configuration.

Node count range * ⓘ 1 5

Create Kubernetes cluster ...

Basics **Node pools** Access Networking Integrations Advanced Tags Review + create

Node pools

In addition to the required primary node pool configured on the Basics tab, you can also add optional node pools to handle a variety of workloads [Learn more about node pools](#)

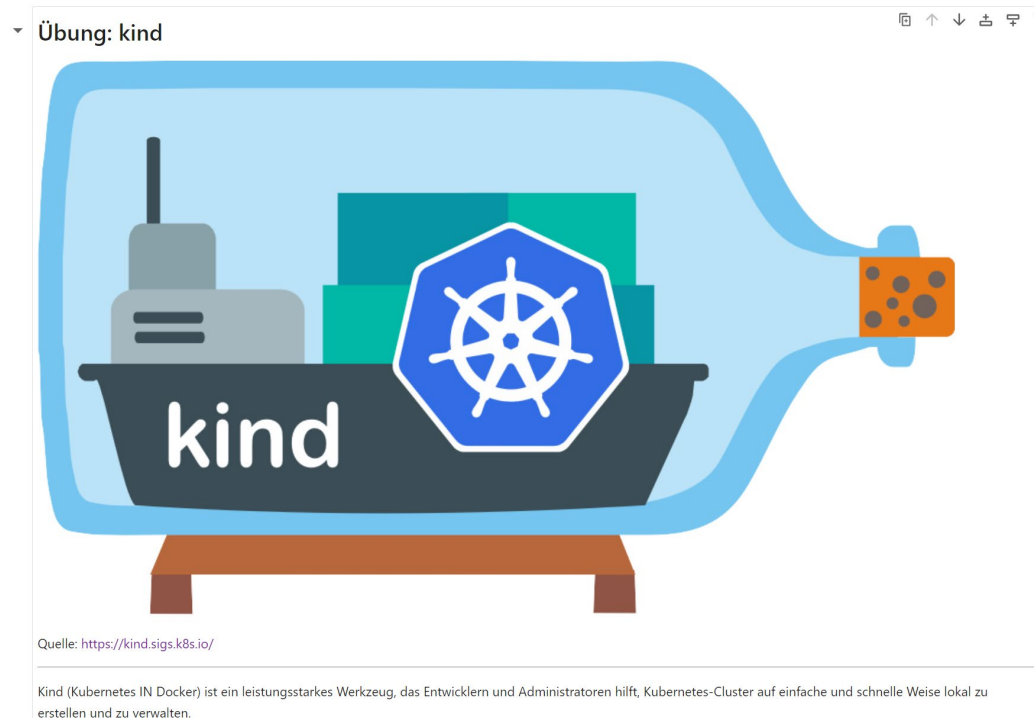
+ Add node pool

Name	Mode	OS type	Node count	Node size
☐ agentpool	System	Linux	1-5	Standard_DS2_v2
☐ winp	User	Windows	1-20	Standard_B2ms



Übung 06-1: Kubernetes Cluster mit kind

- Klickt auf den Link unten und spielt die Übung durch:



- Erkenntnisse:** kind ermöglicht den Betrieb eines kompletten Kubernetes-Clusters in Docker-Containern – ideal für Entwicklung, Schulung und Tests. Zeigt, dass **Kubernetes selbst aus Containern besteht** und somit hochgradig portabel ist. Unterstützt das **schnelle, ressourcenschonende Experimentieren** ohne Cloud- oder VM-Infrastruktur. Demonstriert das **Zusammenspiel von Docker und Kubernetes** auf einfacher, nachvollziehbarer Ebene.
- Hierarchie:**
 - ◇ Docker Container
 - K8s Node (auch Container Host)
 - ◇ Pod (IP-Adresse)
 - Containers (Mehrzahl!)
 - Prozess(e)

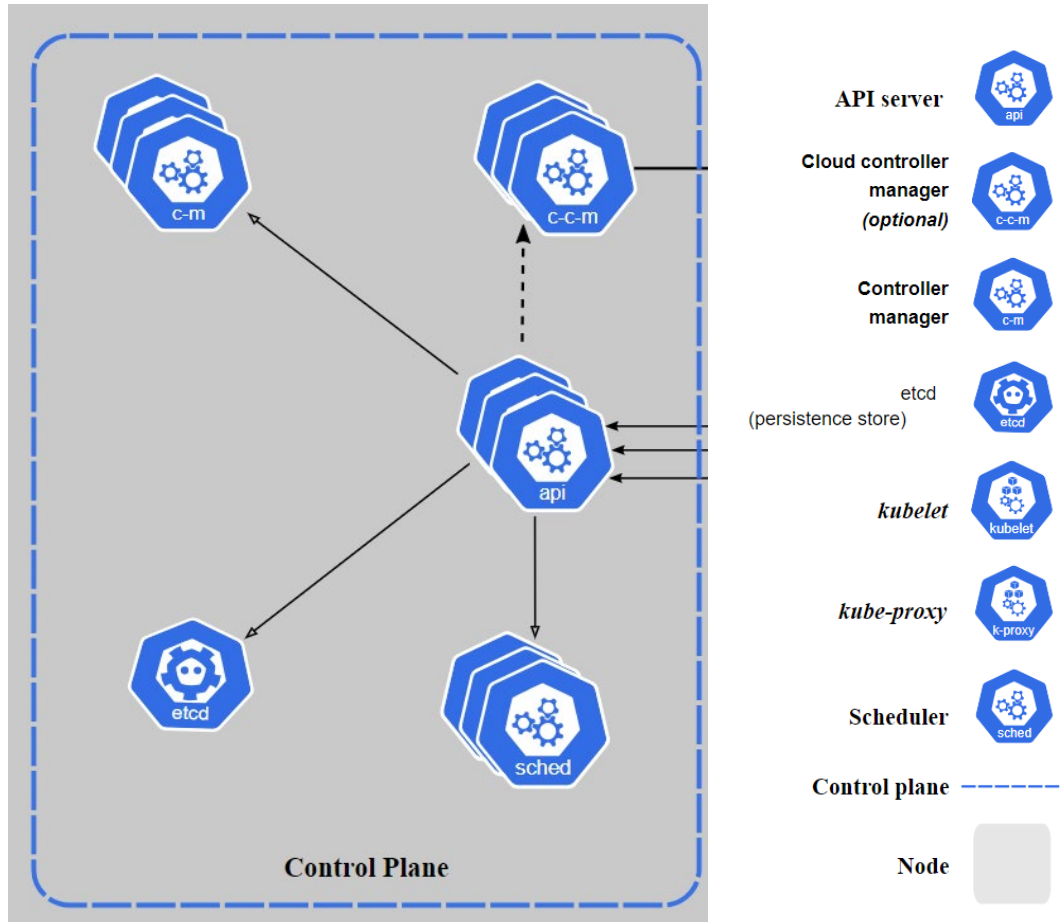
<http://dukmaster-10-default.mshome.net:32188/notebooks/work/06-1-kind.ipynb>

K8s-Dienste auf der Control Plane

```

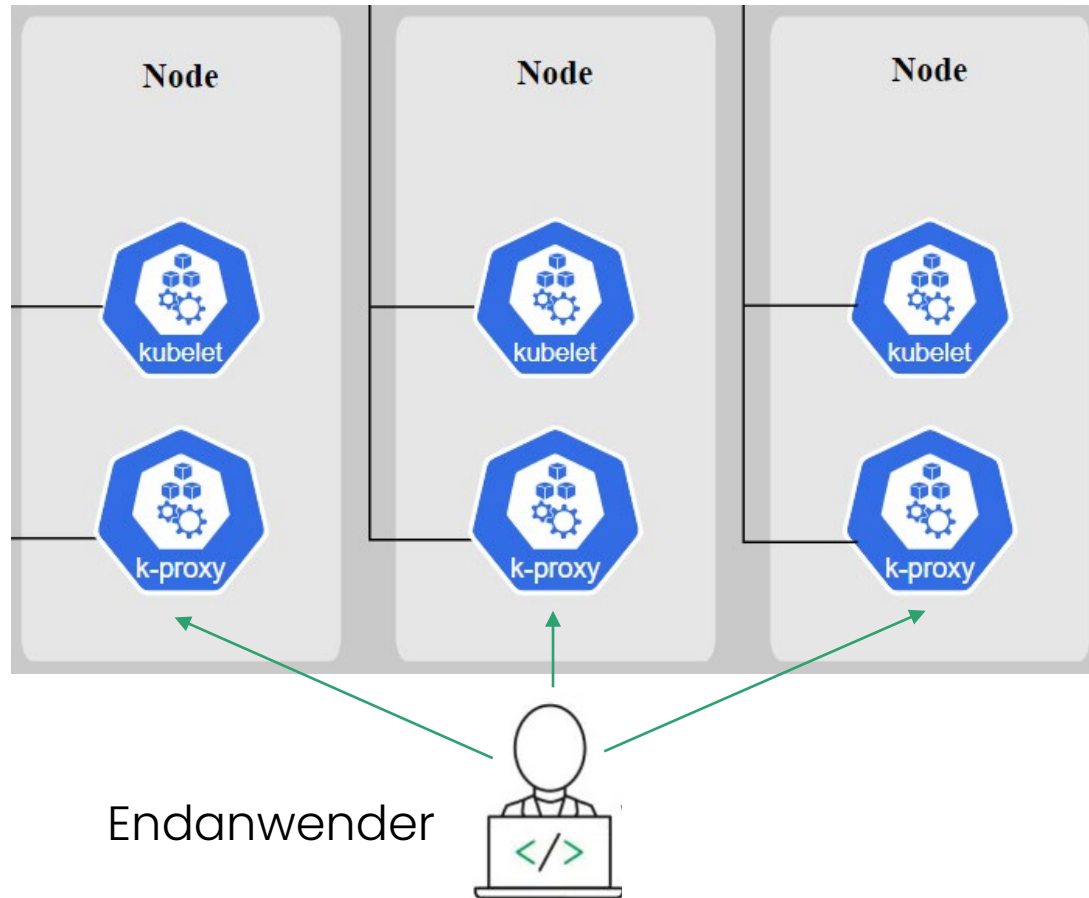
~01:~$ kubectl get pods --all-namespaces
NAME                                READY
coredns-5644d7b6d9-dsxh7           0/1
coredns-5644d7b6d9-ndwjg           0/1
etcd-master-01                      1/1
kube-apiserver-master-01            1/1
kube-controller-manager-master-01  1/1
kube-proxy-j7fb2                    1/1
kube-scheduler-master-01            1/1

```



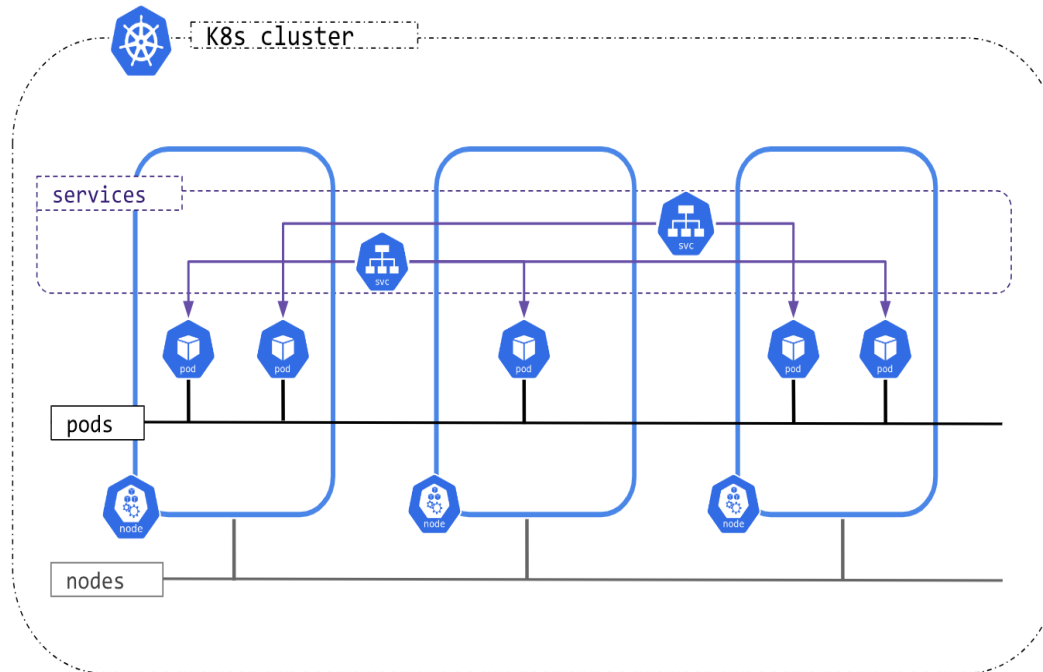
- **API-Server**
 - ◇ zentrale administrative Service-Komponente.
 - ◇ Managen aller Ressourcen des K8s Clusters.
- **Controller Manager (c-c-m, c-m)**
 - ◇ Er ist u.a. verantwortlich für die gewünschte Anzahl von ReplicaSets-Objekten im gesamten Cluster.
- **Scheduler (sched)**
 - ◇ Der Scheduler-Prozess weist den Worker Nodes ihre Workloads, also die Pods/Container-Instanzen zu. Der Scheduler Prozess (**kube-scheduler**) kennt die Auslastung aller Worker Nodes und kann die Pods so immer bestmöglich platzieren.
- **Etcd**: Key/Value (KV)-Store (No-SQL Datenbank)
- **CoreDNS**: DNS-Server

K8s-Dienste auf den Worker Nodes



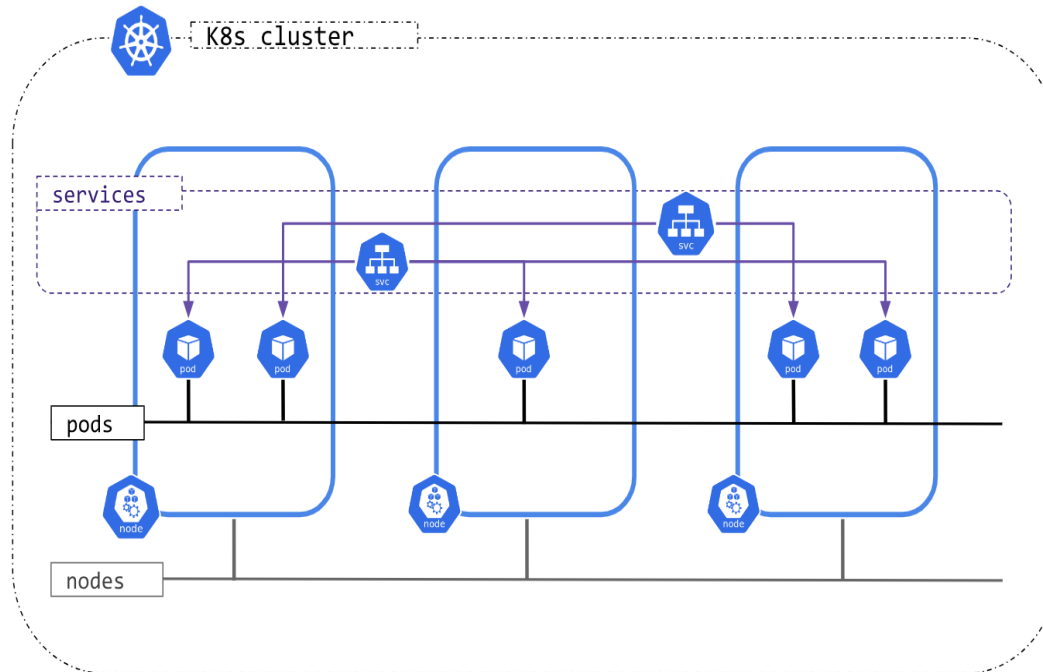
- **Kubelet (Cloud siehe <https://kubernetes.io/docs/concepts/architecture/cloud-controller/>)**
 - ◇ Der kubelet-Service verbindet die Worker Nodes mit dem Kubernetes Control Plane.
- **Kube-Proxy (k-proxy)**
 - ◇ Der K8s-Netzwerk-Proxy – er arbeitet auf jedem Worker Node und kümmert sich um die eingehenden Requests und ihr Routing.
 - ◇ Er kann als primitiver (Layer 4) Loadbalancer fungieren.
- **Container Runtime (Engine)**
 - ◇ Die eigentliche Container-Plattform, mit der K8s arbeitet.

Networking in Kubernetes (1)



- **Kubernetes** verwendet im Unterschied zu Docker eine **flache Netzwerkstruktur**:
 - ◇ Jeder Container kann mit jedem anderen ohne NAT kommunizieren.
 - ◇ Alle Kubernetes Nodes können mit allen Containern (und in die andere Richtung) ohne NAT kommunizieren.
 - ◇ Die IP, die ein Container von sich selbst sieht, ist auch die, die jeder andere Node oder Container im Netz von ihm sieht.

Networking in Kubernetes (2)

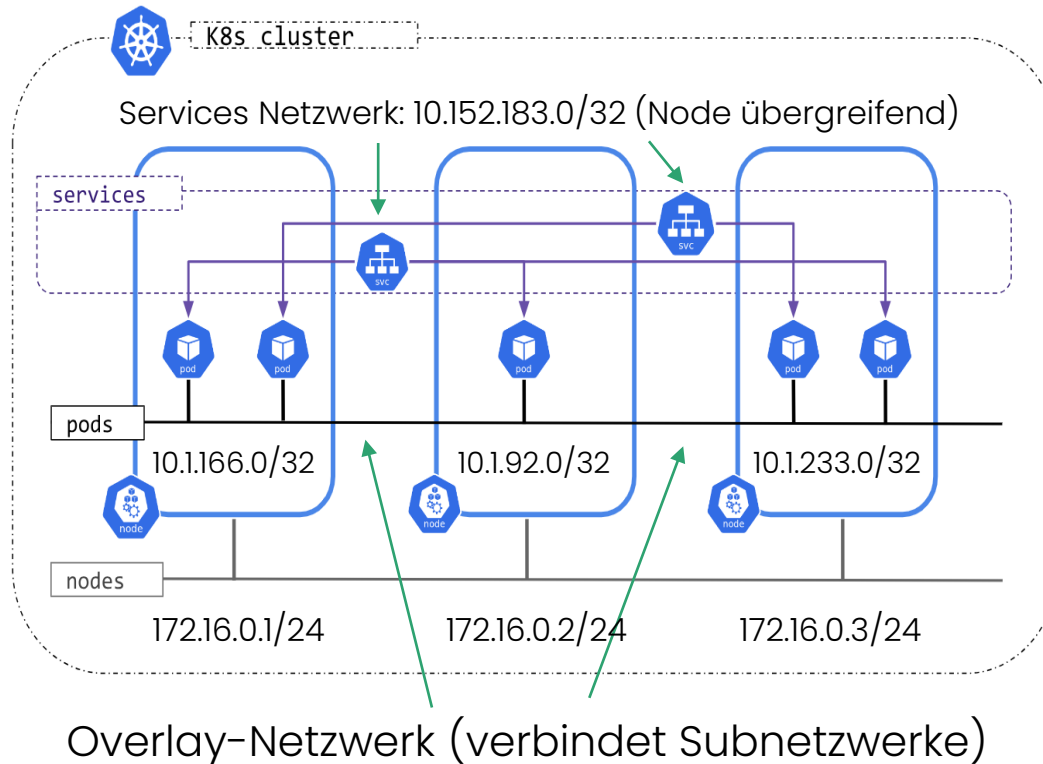


- Grundsätzlich kann man die Kommunikation in K8s-Netzen in fünf (Buch vier) verschiedene Bereiche unterteilen:
 - ◇ **Container-zu-Container-Kommunikation:** Container innerhalb eines Pods teilen sich die gleiche IP.
 - ◇ **Pod-zu-Pod-Kommunikation:** Jeder Pod eines K8s Clusters erhält eine IP.
 - ◇ **Pod-zu-Service-Kommunikation:** ein Service stellt ein DNS-Name/Port zur Verfügung (port-based routing).
 - ◇ **Pod-zu-Ingress-Kommunikation:** ein Ingress stellt ein URL zur Verfügung (Reverse Proxy, URL-based routing)
 - ◇ **Externe-zu-interner-Kommunikation (Egress = Out, Ingress = In):** via externe Loadbalancer, Services, Ingress.
<https://medium.com/@shreysms31/networking-in-kubernetes-130c6ac5b616>



```
r-01:~$ kubectl get pods --all-namespaces
NAME                                READY
coredns-5644d7b6d9-dsxh7           1/1
coredns-5644d7b6d9-ndwjg           1/1
etcd-master-01                      1/1
kube-apiserver-master-01            1/1
kube-controller-manager-master-01  1/1
kube-flannel-ds-amd64-pn94s        1/1
kube-proxy-j7fb2                   1/1
kube-scheduler-master-01           1/1
```

Networking in Kubernetes (3)

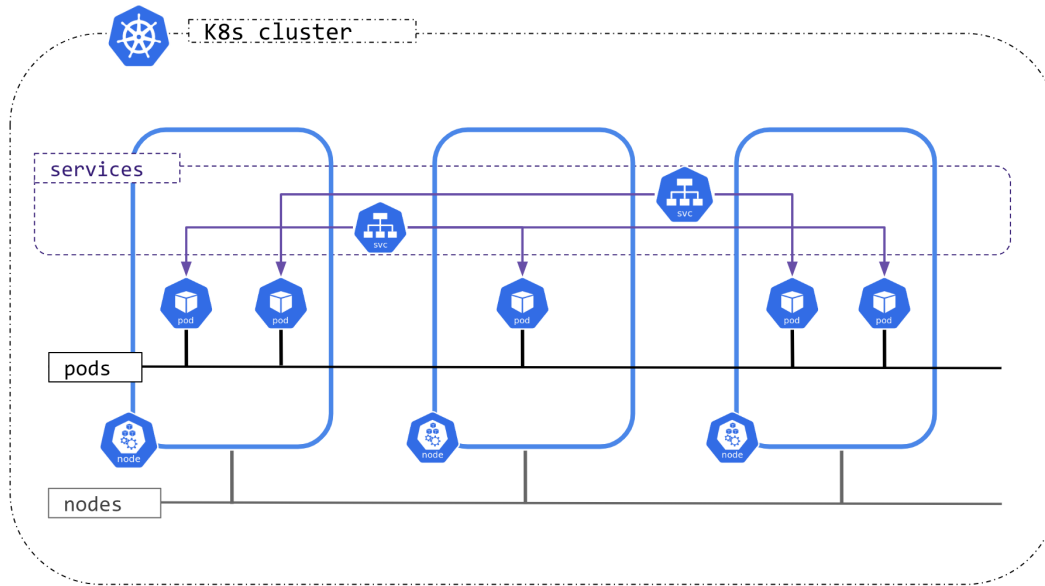


- Was K8s zunächst fehlt, ist ein **Overlay-Netzwerk** welches die Pods **Node-übergreifend** für die **Interkommunikation** nutzen können.
- Das Overlay-Netzwerk wird zwischen jedem K8s Node eingerichtet, sodass sich die Container/Pods auf verschiedenen Nodes direkt sehen bzw. miteinander kommunizieren können.
- **Flannel** ist solch ein Overlay-Netzwerk, stammt aus der CoreOS-Schmiede, und fungiert als speziell für K8s designedes Netzwerk-Overlay.
- Weitere Informationen:
 - ◇ [How to implement the Kubernetes networking model](#)
 - ◇ [CNI - the Container Network Interface](#)
 - ◇ [5G Core Deployment](#)
- Die Kurs-Umgebung verwendet das Overlay-Netzwerk [calico](#).



Übung 06-3: Overlay Netzwerk

- Klickt auf den Link unten und spielt die Übung durch:



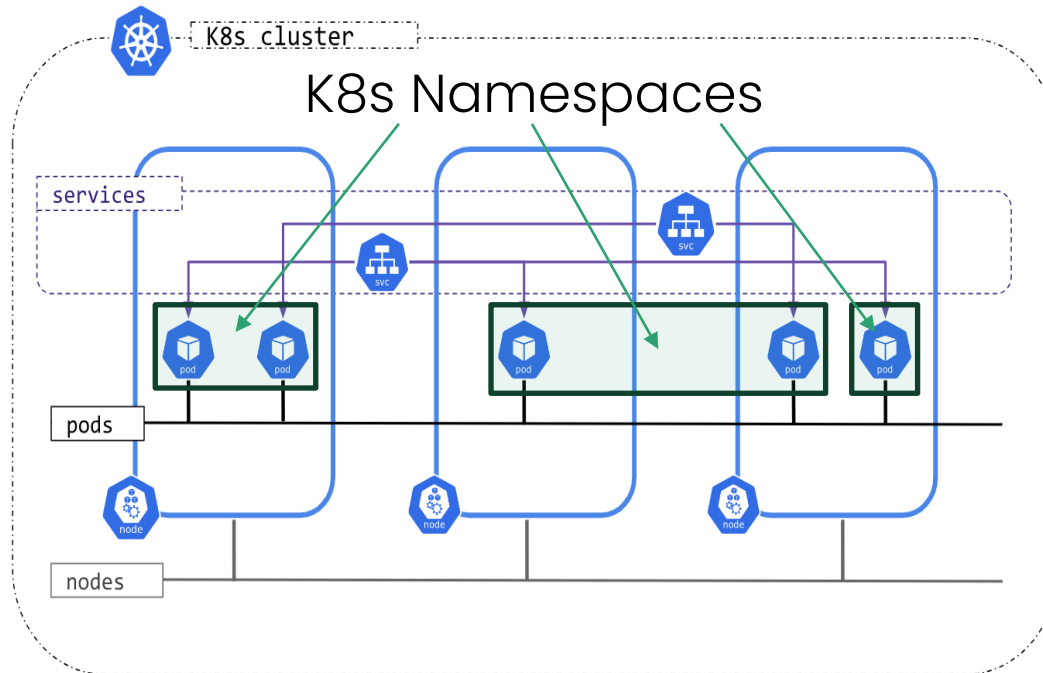
Quelle: Kubernetes

Ein Kubernetes Overlay Network ist eine Netzwerktechnologie, die es ermöglicht, Container in einem Kubernetes-Cluster über verschiedene physische oder virtuelle Netzwerke hinweg zu verbinden.

Diese Netzwerke abstrahieren die physische Netzwerkstruktur und bieten eine einheitliche Kommunikationsschicht für die Pods innerhalb des Clusters.

<http://dukmaster-10-default.mshome.net:32188/notebooks/work/06-2-OverlayNetwork.ipynb>

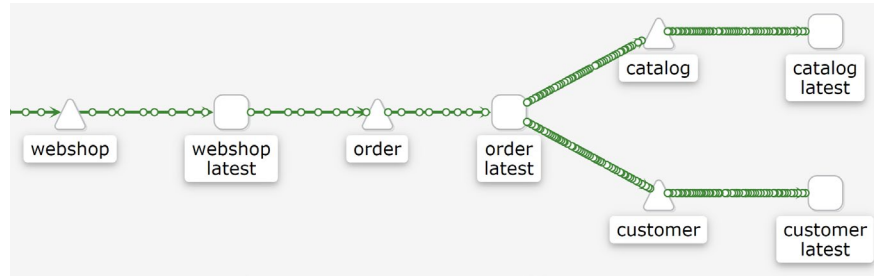
Networking in Kubernetes (5): Namespaces



- Unterteilen den gesamten K8s Cluster in logische Partitionen bzw. Bereiche.
- Standard Namespaces
 - ◇ **default:** In diesem Namespace werden alle neuen erzeugt, sofern wir nichts anderes angeben.
 - ◇ **kube-system:** Kubernetes-Systemdienste.
- Ressourcen in Namespaces können limitiert werden (Anzahl Pod's, CPU, Memory etc.).
- Der Netzwerktrafik kann, mittels [Network Policy Provider](#), eingegrenzt werden
- **Achtung:** Wird ein Namespace gelöscht, werden damit auch alle in ihm gehosteten Objekte/Ressourcen gelöscht!

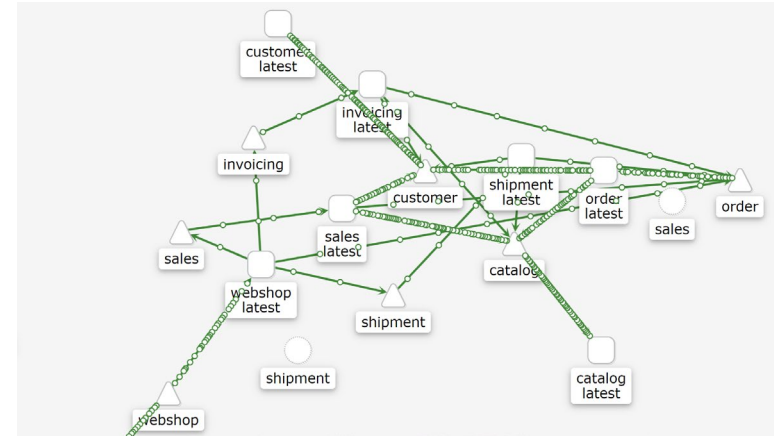
Networking in Kubernetes (6): Beispiel Microservices

- Synchroner Microservices (REST)
 - ◊ Namespace: ms-rest



- Flache Netzwerkstruktur (auch bei unterschiedlichen Nodes)
 - ◊ `ping order || catalog || customer -> OK`
- Namespace (anderer Namespace)
 - ◊ `ping invoicing -> schlägt fehl`
 - ◊ `ping invoicing.ms-brkr -> OK`

- Asynchrone Microservices (Messaging)
 - ◊ Namespace: ms-brkr



- ◊ `ping order -> nur innerhalb Namespace`
- ◊ `ping order.ms-rest -> OK, aber anderer NS`



Übung 06-3: K8s Namespace und Mandanten

- Klickt auf den Link unten und spielt die Übung durch:

▼ Übung: Kubernetes Namespaces und Verschiedene Mandanten (Tenants)



Kubernetes Namespaces sind ein zentrales Konzept in der Verwaltung und Organisation von Ressourcen innerhalb eines Kubernetes-Clusters. Sie bieten eine logische Trennung, die es ermöglicht, z.B. verschiedene Mandanten (Tenants) unabhängig voneinander zu betreiben.

Die Verwendung von Namespaces bringt mehrere Vorteile mit sich:

- Ressourcentrennung: Namespaces isolieren die Ressourcen verschiedener Anwendungen oder Teams. Dies bedeutet, dass Konflikte vermieden werden, beispielsweise wenn mehrere Anwendungen den gleichen Ressourcennamen verwenden.
- Zugriffskontrolle: Durch die Verwendung von Role-Based Access Control (RBAC) in Kombination mit Namespaces können Administratoren den Zugriff auf Ressourcen spezifisch für jeden Namespace regeln. So kann sichergestellt werden, dass ein Team nur auf seine eigenen Ressourcen zugreifen kann.
- Ressourcenquoten: Namespaces ermöglichen es, Ressourcenquoten (z.B. für Memory, Anzahl Container etc.) festzulegen, die sicherstellen, dass ein Team oder eine Anwendung nicht mehr Ressourcen als vorgesehen verbraucht. Dies hilft, eine faire Ressourcennutzung im Cluster zu gewährleisten.
- Einfache Verwaltung: Namespaces vereinfachen die Verwaltung und Überwachung von Ressourcen, indem sie eine klare Trennung und Strukturierung bieten. Dies ist besonders in großen Clustern mit vielen Anwendungen und Teams von Vorteil.

In nachfolgenden Beispiel erstellen wir drei Namespaces, für jeden Mandanten einen und starten den Webshop in unterschiedlichen Versionen.

<http://dukmaster-10-default.mshome.net:32188/notebooks/work/06-3-KubernetesNamespaces.ipynb>



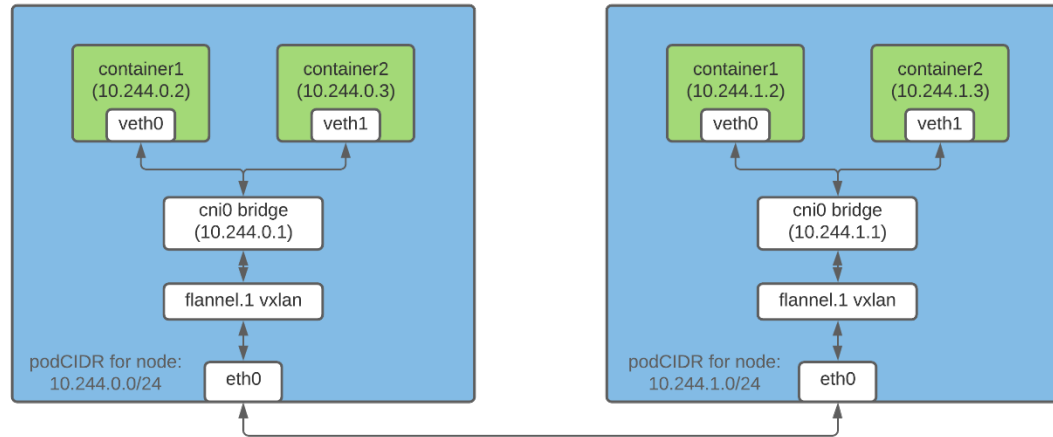
Reflexion

- Das im Juli 2014 gestartete Kubernetes (griechisch: Steuermann) stellt die derzeit **populärste** Container-Cluster-/Orchestrierungs-Lösung dar.
- Von Single Node über Cluster, zu HA-Cluster zu Federated K8s werden praktisch alle Setups unterstützt.
- Das Container Ökosystem umfasst alle wichtigen Technologiefirmen und wird durch die Open-Source-Software-Stiftung «Cloud Native Computing Foundation» (CNCF) gefördert.
- Es gilt heute, Dank der [CNCF](#), als **Industrie Standard**.

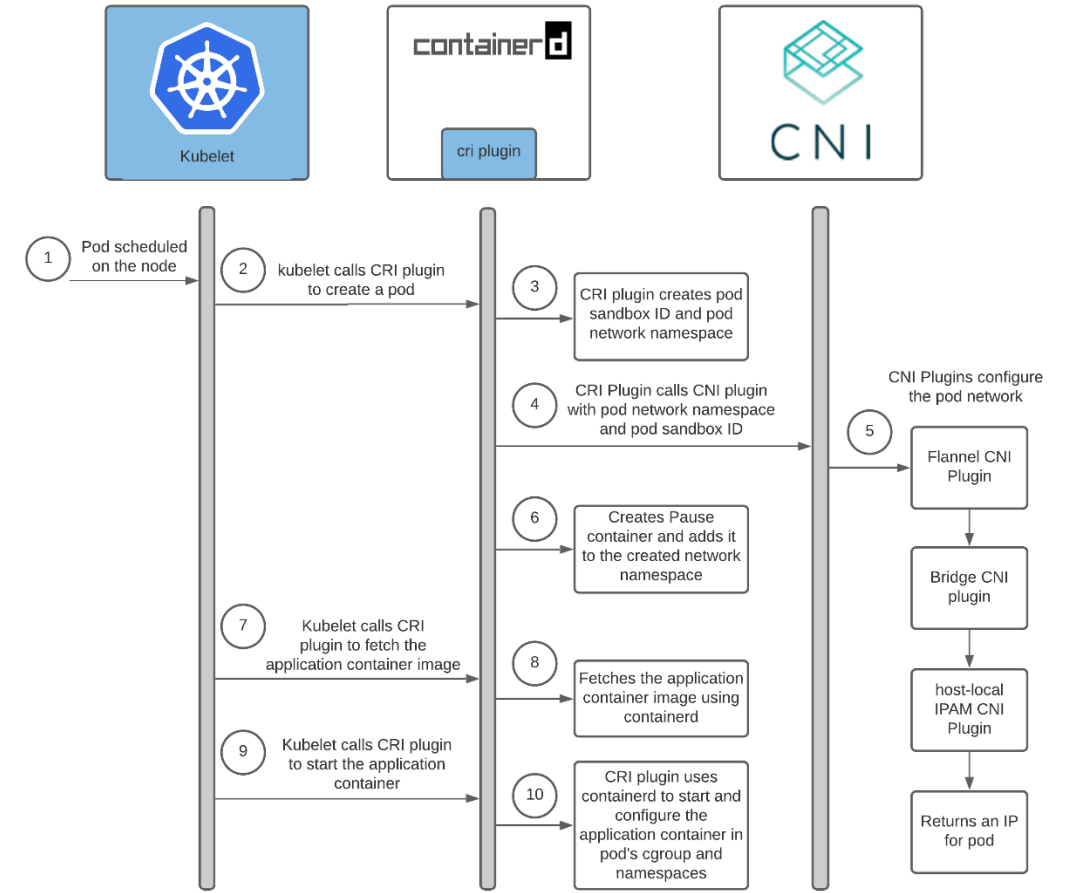
Lernzielkontrolle

- Sie haben einen ersten Einblick in Kubernetes
- Sie haben einen ersten Überblick über das Container Ökosystem.

Networking in Kubernetes (4)



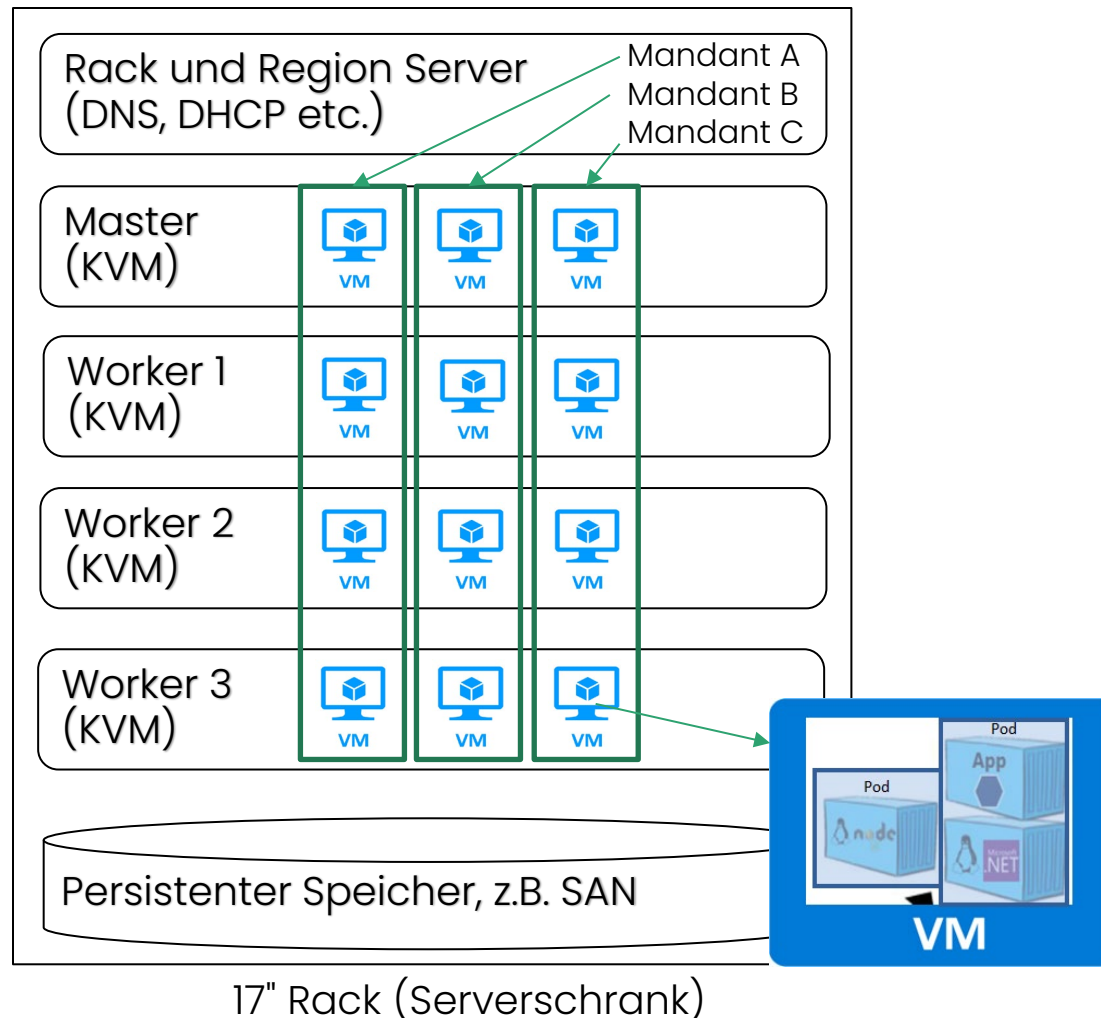
- Container im gleichen Worker: vethX auf Linux Bridge.
- Container auf verschiedenen Workern: z.B. Flannel vxlan und Routing Table.



Beispiel Zalando: mehrere Cluster und gemeinsam Monitoren

- Kubernetes ist, in der Vergangenheit, nicht bekannt für seine Mandantenfähigkeit. "Kleinere" Cluster können einen Teil dieses Problems mindern.
- Zalando hat 80 Kubernetes Cluster, in der AWS Cloud, aufgesetzt und nutzt einige Infrastrukturen gemeinsam, z. B. Monitoring (Prometheus) und der Ingress-Proxy ([Skipper](#)).
- Der Ausfallradius ist begrenzt – alles, was in einem Cluster schief geht (Ausfall, Sicherheitsvorfall usw.), wirkt sich nicht unbedingt auf die gesamte Organisation aus.
- Die Kostenaufteilung ist einfacher (jeder Cluster gehört zu einer Kostenstelle).
- Der Cluster (und sein AWS-Konto) dienen als natürliche Vertrauensgrenze für die Zugriffssteuerung.

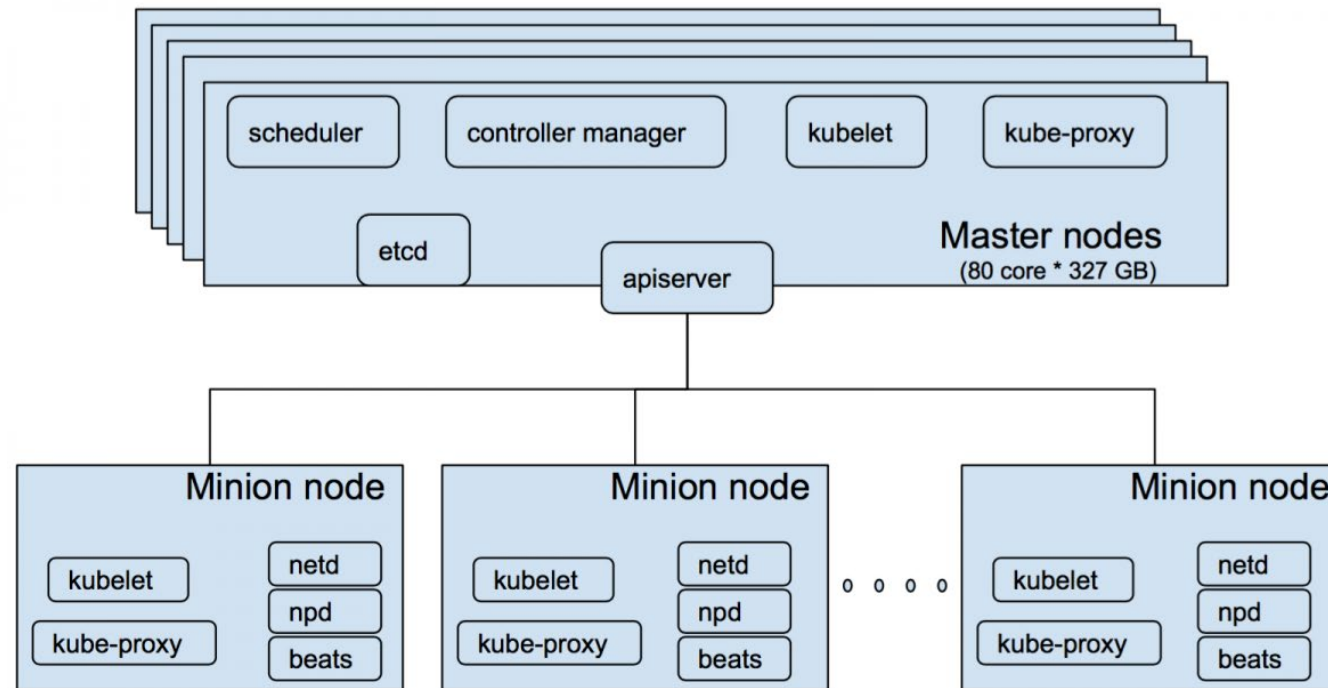
Beispiel: Kubernetes Cluster für 3 Mandanten



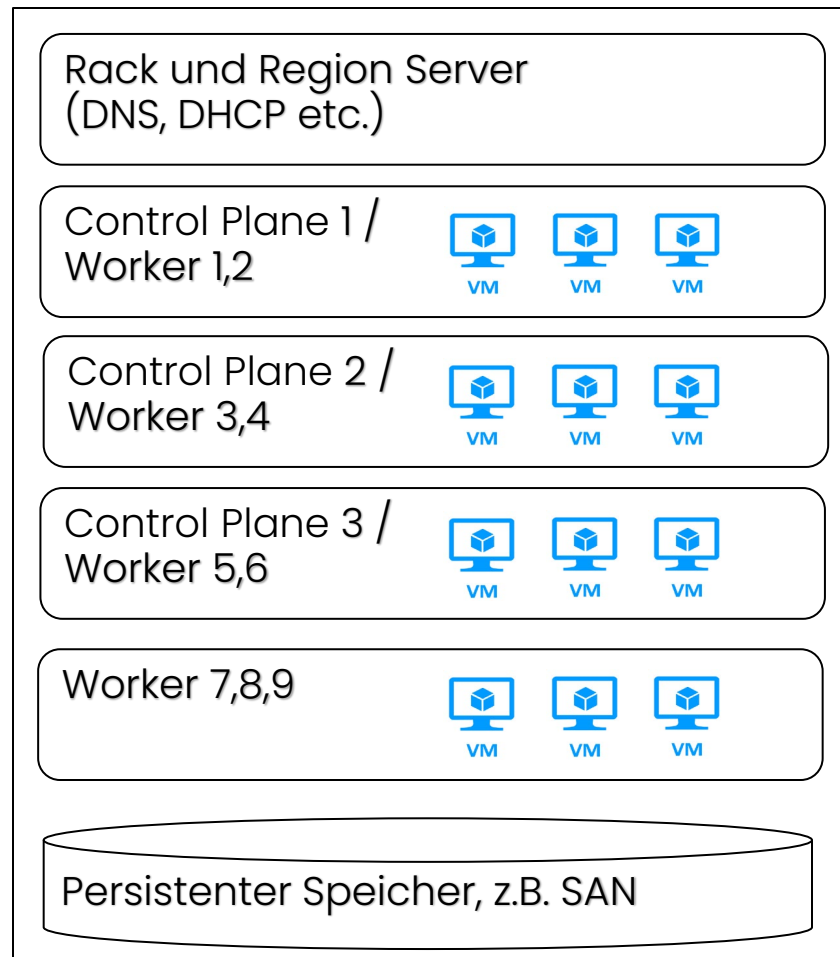
- Es werden mindestens 3 Nodes pro Cluster installiert.
- Die Nodes werden über die physikalischen Server verteilt.
- Es wird eine Kubernetes Distribution verwendet, welche keine Unterscheidung Control Plane/Worker Nodes macht.
- **Hinweis:** durch Aufteilung von Kubernetes auf mehrere VMs, kann im laufenden Betrieb, ein Update der K8s Nodes durchgeführt werden.

Beispiel ebay: Ein Cluster aus 5000 Nodes

- Kubernetes behauptet das 5.000 Nodes in einem Cluster möglich sind.
- Der eBay Tess.IO-Cluster bestätigt, nach der Optimierung / Korrektur, diese Behauptung.



Beispiel: Kubernetes Cluster für 3 Mandanten



17" Rack (Serverschränk)

- Es wird ein grosser Kubernetes Cluster eingerichtet.
- Es werden mehrere Control Plane, verteilt über die physikalischen Maschinen erstellt.
- Die Worker Nodes werden über die physikalischen Maschinen verteilt.
- Die Unterteilung nach Mandanten erfolgt mittels Kubernetes Namespaces.
 - ◇ K8s Namespace: MA
 - ◇ K8s Namespace: MB
 - ◇ K8s Namespace: MC

Sind Geografisch verteilte Cluster möglich?

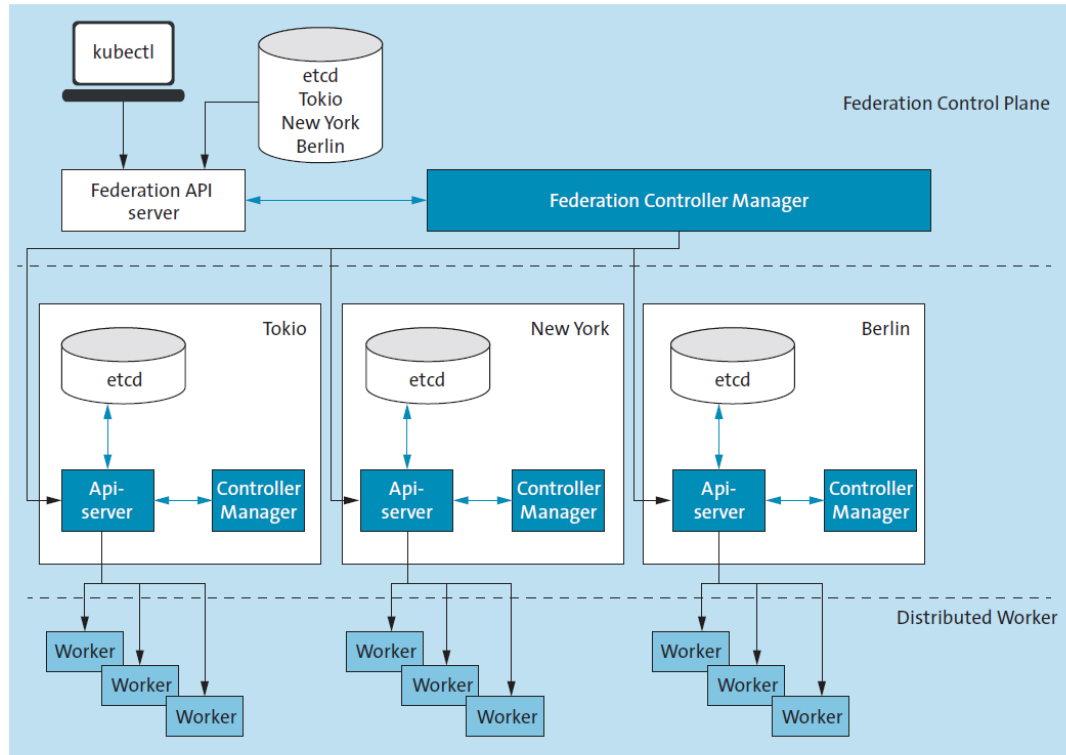


Abbildung 17.1 Federated K8s Cluster

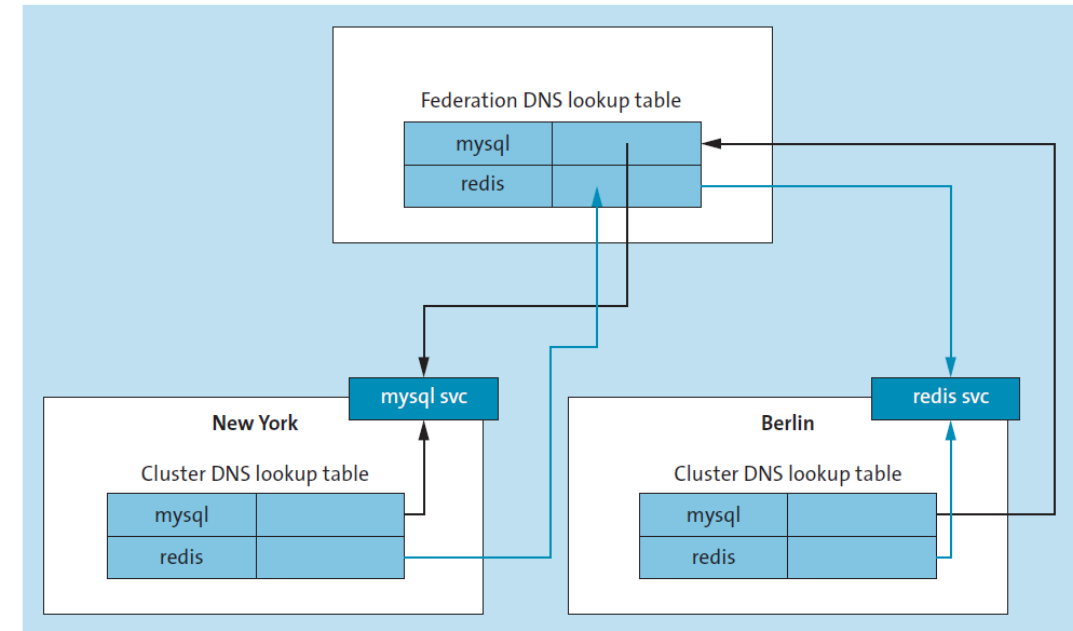
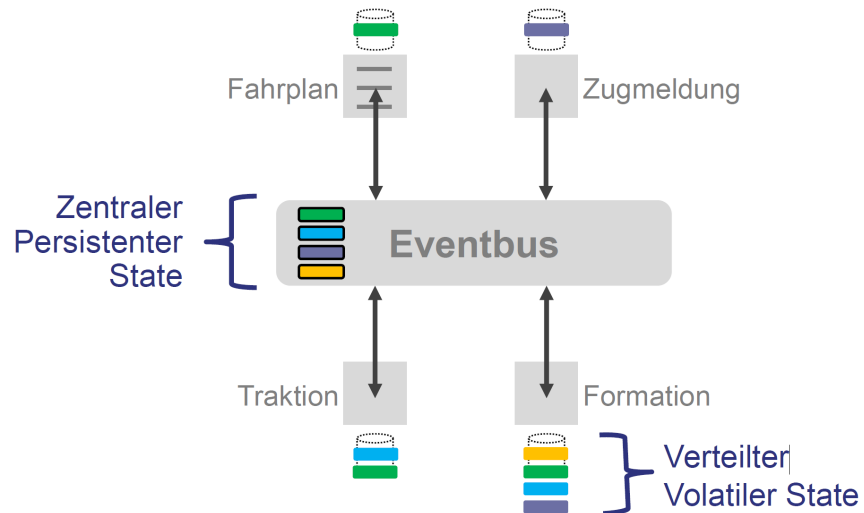


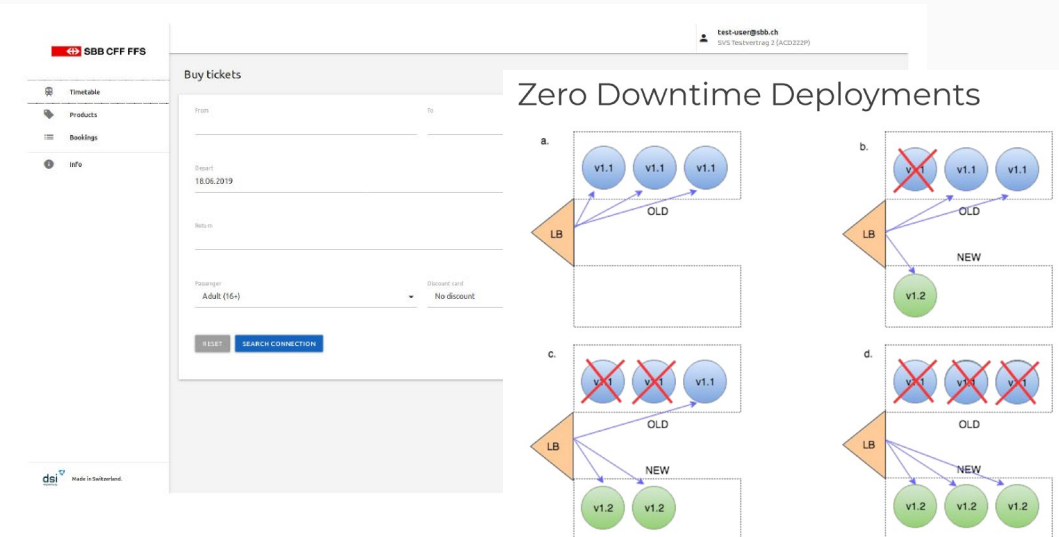
Abbildung 17.2 DNS mit Federated Setup

SBB (Messaging und K8s Worker Updates)



- SBB-Güterverkehr
- Quelle: Event-basierte Architektur mit Apache Kafka:

https://www.jug.ch/html/events/2019/architektur_mit_apache_kafka_bs.html



- Agenturclient: Allows SBB business customers to sell tickets
- Quelle: Real World Kubernetes:

https://www.jug.ch/html/events/2019/kubernetes_live_zh.html

Gibt es Fallstudien für Kubernetes?

End User Case Studies

CNCF end users are telling their stories to help elevate the technical conversations to business objectives and challenges. These are real-world use cases about the impact cloud native projects are having on their business. Click on a box to read how that end user has used cloud native technologies and CNCF projects to help their business.

Filter the list of case studies:

All Industries

All Cloud Types

All Projects

All Challenges

All Countries

All Product Types

Reset filter

Displaying 64 case studies

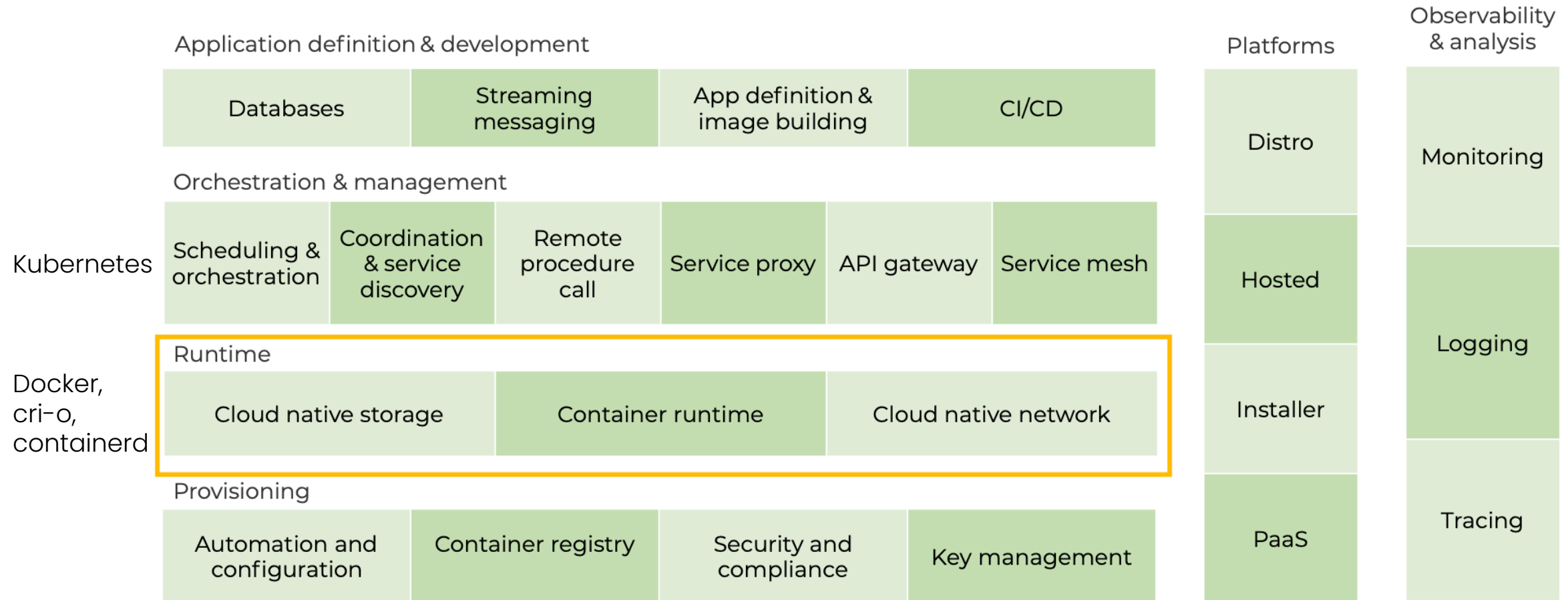
The main grid displays 12 case study cards, each with a company name and a 'READ THE CASE STUDY' button. The companies shown are Adform, Adidas, AlphaSense, Amadeus, Ancestry, Ant Financial, CERN, ricardo.ch, AppDirect, BlaBlaCar, BlackRock, and Bloomberg. An inset window shows a filtered view of the case studies, specifically for Switzerland, displaying 2 of 64 case studies. The inset also includes the same filter controls as the main page.

<https://www.cncf.io/newsroom/case-studies/>

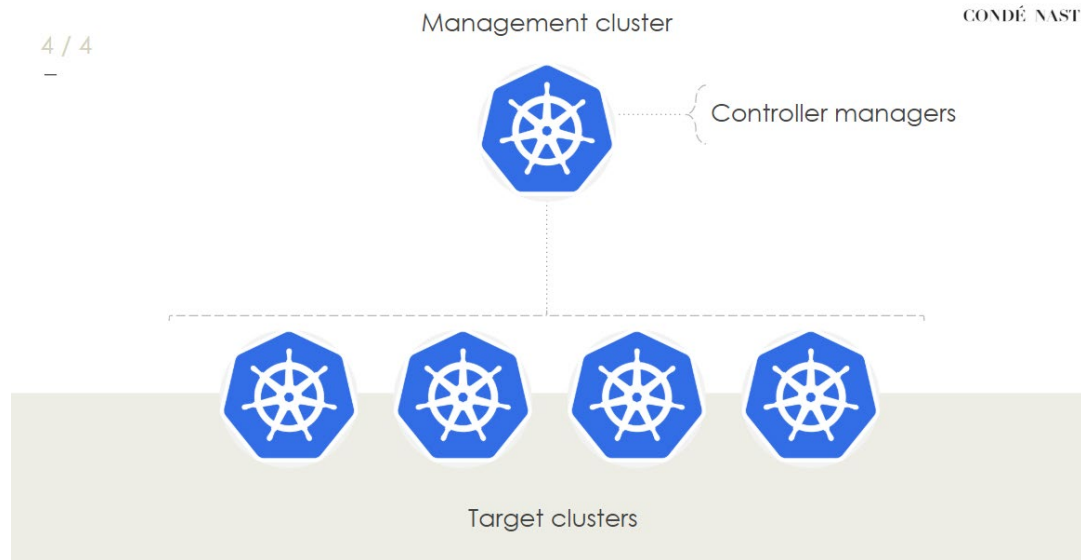
Gibt es Schweizer Fallstudien?

- **CERN**
 - ◇ Seit **Februar 2018** im Einsatz
 - ◇ Kubernetes für die Orchestrierung, Helm für die Bereitstellung, Prometheus für die Überwachung und CoreDNS für die DNS-Auflösung innerhalb der Cluster
 - ◇ Hat erlaubt **hybride Cloud-Strategie** einzuführen
 - ◇ 330 Petabyte an Daten, diese können sich in den nächsten Jahren um das Zehnfache erhöhen
 - ◇ Die Zeit für die Bereitstellung eines **neuen Clusters** für ein komplexes verteiltes Speichersystem ist von mehr als 3 Stunden auf weniger als **15 Minuten** gesunken ...
- **Ricardo**
 - ◇ Erster **Cluster im 2016**
 - ◇ Die Migration ist zur Hälfte abgeschlossen und das Unternehmen plant, bis Ende 2018 vollständig auf Google Cloud Platform umzustellen
 - ◇ Kubernetes für die Clusterverwaltung, Prometheus für die Überwachung und Fluentd für die Protokollierung
 - ◇ Die Aufteilung des Monolithen in Microservices ermöglichte eine höhere Geschwindigkeit neue Services bereitzustellen und **Kubernetes** war **entscheidend** bei der Umstellung
 - ◇ Das klassische Problem von erstellen, bearbeiten und ausliefern von neuen Versionen, konnte durch **30 - 60 Auslieferungen pro Tag statt 10 pro Woche** gelöst werden

Container Runtime Layer



Wie Installiere ich einen Multicluster?



- **ClusterAPI** bietet einen deklarativen Satz von APIs zum Erstellen, Konfigurieren, Verwalten und Löschen von Clustern.
- Mit diesem Tool soll eine einheitliche und nachhaltige Schnittstelle für die Cluster-Initialisierung vor Ort und unterstützten Cloud-Anbietern bereitgestellt werden.
- ClusterAPI ist derzeit in der Version **v1alpha2** und lässt sich mit 12 großen Infrastrukturanbietern integrieren.
- <https://cluster-api.sigs.k8s.io/>

Wie ist die Sicht von Dev vs. Ops auf K8s?

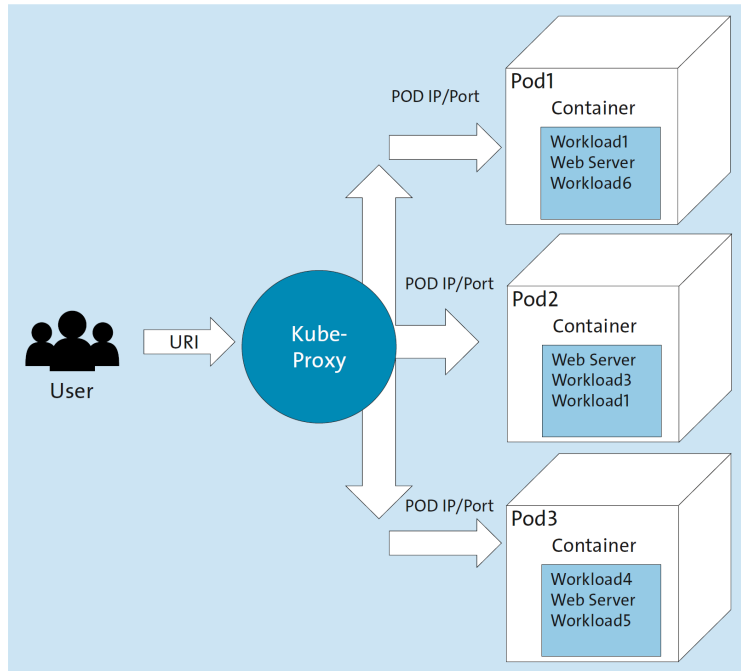
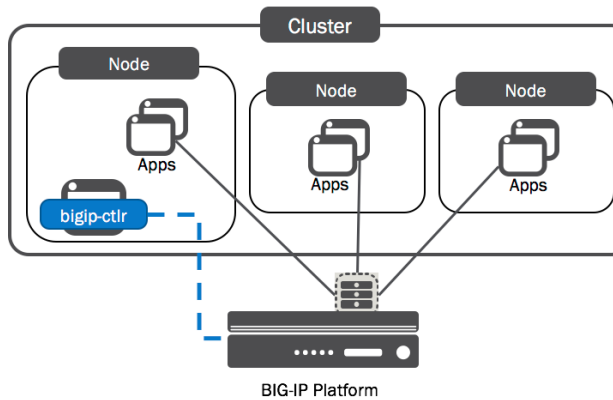


Abbildung 13.2 Arbeitsweise des Kube-Proxy exemplarisch auf einem K8s Worker Node



User Guides

- [BIG-IP and flannel VXLAN Integration](#)
- [Add BIG-IP device to flannel VXLAN](#)
- [Install the BIG-IP Controller: Kubernetes](#)
- [Managed BIG-IP objects](#)
- [Attaching Virtual Servers to Services](#)
- [Using the BIG-IP Controller as an Ingress controller](#)
- [CIS and AS3 Extension integration](#)
- [Manage your BIG-IP virtual servers](#)
- [F5 Resources Explained](#)
- [Deploy iApps with the BIG-IP Controller](#)
- [Troubleshoot Your Kubernetes Deployment](#)

- Anwendungsorientiertes statt Technisches Denken.
- Abstraktion der Infrastruktur statt in Rechnern Denken.

Worker-Node: Scheduling and Eviction

- Kubernetes bietet eine Vielzahl von Möglichkeiten um Pods (Container) auf den Worker Nodes zu platzieren.
- Siehe dazu das Kapitel [Scheduling and Eviction](#) in der Kubernetes Dokumentation.
- Dort finden sich auch Informationen um das Räumen und neu Starten von Pods (Containers), wenn eine Worker Node nicht mehr verfügbar ist, fein zu tunen.
- Meistens fehlt der Eintrag `--pod-eviction-timeout=30s` in der Datei `/etc/kubernetes/manifests/kube-controller-manager.yaml` oder ist auf 300s (10 Minuten) gesetzt.
- Nach Ändern der Datei wird automatisch der Controller Manager frisch gestartet und die Werte übernommen.